Givin.
thoughtful gifting, in your network

Build Plan & Technical Strategy

An honest, end-to-end plan for building Givin: a way to send real digital gifts to professional connections at the moments that matter. Optimized for the lowest possible cost and for spreading virally. Written as a build reference for you and your dev team.

VERSION 1.0 JUNE 2026 INTERNAL — PRE-PATENT

Do this first: rotate your Stripe key

Your planning PDF contains a **live** Stripe secret key (`rk_live_…`). Anyone holding that document can charge through your account. Log into Stripe → Developers → API keys → **roll/revoke it now**. Never place secrets in documents, screenshots, or a git repo again — they belong in environment variables and a secrets manager only.

The short version

My honest recommendation, before the detail:

1. **Build the web app first, the extension second.** A standalone app at `givin.app` is fully in your control. The Chrome extension is a thin "launcher" that hands off to it — not the product itself.
2. **Do not depend on LinkedIn's API.** The data and messaging you'd want (contacts, auto-DMs) is partner-gated and effectively closed. Treat LinkedIn as a place where the moment happens, not as a backend.
3. **Deliver gifts by email + a redeem link**, not "inside LinkedIn." The recipient chooses what they want and where to redeem — no shipping address needed.
4. **The free gift is your growth engine, not a feature.** Recipients become senders. Cap it, fund it cheaply, and consider letting brands sponsor it.
5. **Start at near-zero monthly cost** (Supabase + Vercel + Tremendous + Resend) and only pay per gift actually sent. Don't over-invest in the patent — your real moat is brand, speed, and the network loop.

01 The honest assessment

This is a genuinely good idea. Professional gifting is underserved, the moments are real and frequent, and "keep and grow relationships" is exactly what LinkedIn users already try to do badly with a generic "Congrats!" comment. But two things in the original plan will hurt you if you build them literally:

WHAT'S STRONG

- Clear, recurring trigger moments
- Low-friction, high-emotion product
- Built-in virality (every gift reaches a new person)
- Multiple revenue paths (margin, teams, sponsors)

WHAT'S HARD

- LinkedIn's API won't give you contacts or messaging
- A DOM-injecting extension risks user account bans
- "Deliver via LinkedIn messaging" isn't feasible self-serve
- The patent is weak; don't let it slow you down

The plan below keeps everything that's strong and routes around everything that's hard.

02 What Givin actually is

One sentence: **Givin lets you send a real gift to a professional contact in under 30 seconds — the recipient picks what they want and where to redeem it.**

The product is three jobs done well:

- **Notice the moment** — a new job, a work anniversary, a great meeting, a closed deal.
- **Send something real** — flowers, a coffee, chocolate, or a gift card — with a short personal note.
- **Let them redeem easily** — by email + link, choosing their own reward and merchant. No address, no awkwardness.

The trigger moments from your research, ranked by how often they happen and how easy they are to act on:

Moment	Frequency	How you detect it
New job / promotion	Very high	Visible on profile / extension
Work anniversary	High	User adds date / extension reads it
After a meeting / call	High	Manual (calendar sync later)
Closed deal / referral	Medium	Manual
Birthday / holidays	Seasonal	User-added or calendar

Reality: there is no reliable LinkedIn webhook for these. Detection is the user's eyes (in the extension) plus dates the user saves in Givin — not an automated LinkedIn feed.

03 The big decision: extension vs. web app

This is the single most important architectural choice, so let me be very direct about it.

The LinkedIn API reality

LinkedIn's self-serve developer program gives you essentially one useful thing: **"Sign In with LinkedIn using OpenID Connect"** — secure login that returns a member's name, email, and photo. That's it for most apps. The **Connections API, Messaging API, and profile-data APIs are partner-only**, granted through programs (Marketing/Sales partners) that are slow, selective, and not designed for a gifting startup. Assume you will not get them. So:

- You **cannot** import a user's LinkedIn connections via API.
- You **cannot** send a gift "through LinkedIn messaging" via API.
- You **can** use LinkedIn purely as a clean, trusted login.

The browser-extension reality

The yellow mockup — a panel injected onto the LinkedIn page that reads the profile and sends — is the most magical version and the riskiest. LinkedIn's User Agreement prohibits scraping, automated access, and overlaying/altering the service. An extension that bulk-reads connections or auto-sends messages can get **your users' accounts restricted or banned**, which would be fatal to trust. The safe way to ship an extension is to make it deliberately dumb:

THE "COMPANION LAUNCHER" RULE

The extension only acts on a **single explicit click**, on the profile the user is already looking at. It reads just the visible name and headline, then opens Givin pre-filled. It never crawls connections, never runs in the background, never sends a message inside LinkedIn. It's a "Share to Givin" button — not an integration. This keeps it on the right side of the line and keeps your users safe.

Recommendation: phased, web-app-first

Phase A · Web app

The whole product lives at givin.app. Login with LinkedIn, pick a gift, add the recipient's email, send. Self-contained, zero platform risk.

Phase B · Thin extension

A "Send a gift" button on profiles that pre-fills the web app. Pure convenience layer. Easy to ship once the app works.

Phase C · Teams / CRM

Calendar sync, saved dates, team budgets, analytics. This is where the real revenue is.

04 Recommended architecture

Deliberately boring and cheap. Every box below has a generous free tier and is a skill your dev team likely already has.

The recipient never needs an account to receive — they click the link in the email and redeem. They only sign up if they want to send a gift back, which is exactly the loop you want.

05 Tech stack — superslim cost

Layer	Pick	Why	Start cost
Frontend	React + Vite + Tailwind	Fast, standard, hireable	€0
Hosting	Vercel / Cloudflare Pages	Free tier, instant deploy	€0
Backend + DB + Auth	Supabase	Postgres, Auth (incl. LinkedIn OIDC), Edge Functions, RLS — one tool. This is what "Lovable Cloud" is under the hood.	€0
Gift fulfillment	Tremendous	One API, global catalog, free sandbox, you pay only when a gift is redeemed — no upfront, no inventory	€0 + per gift
Email delivery	Resend	3k emails/mo free, great DX	€0
Payments	Stripe	Only when you charge for paid gifts/plans	~1.5% + fee/txn
Extension	Chrome MV3 (vanilla)	Tiny content script; €5 one-time dev account	€5 once

Why not "Lovable" specifically: it's a fine way to prototype fast, and it's already generated a starting point for you. But for something you'll own, patent-protect, and scale, have your team own the code in your own repo with Supabase directly. You can lift the good parts of the Lovable build over.

On gift providers

You listed Tremendous, Tillo, Huuray, GoGift, Tango Card, Awardit. For a superslim, global start, **integrate Tremendous first** — it's the lowest-friction (sandbox, pay-per-redeem, huge catalog including Nordic options). Add **GoGift or Awardit** later for better Nordic margins once you have volume to negotiate. Don't integrate all six; one good integration ships the product.

06 The free tier & unit economics

You said: free gifting up to a level, lowest cost possible, viral. Here's how to make "free" not bankrupt you.

- **Free tier = a small, real reward**, e.g. a €2–4 coffee or a digital card, capped at **~3 sends per user per month**.
- **Each free gift is your customer-acquisition cost.** €2–4 to put your brand in front of a high-value professional and pull them into the loop is cheap CAC.

- **Make sponsors fund it.** A coffee chain or flower brand pays to be the "free gift," reaching professionals at happy moments. This can make the free tier cost-neutral or profitable — it's the strongest version of the model and worth designing for early.
- **Monetize the rest:** margin on paid gifts (5–15% from the provider), team plans (saved dates, budgets, analytics), and "boost" upgrades (bigger gift, gift now).

Guardrail: rate-limit free gifts per sender and per recipient, require a verified login to send, and watch for abuse. Free + viral + money attracts fraud; build the limits in from day one.

07 The viral growth engine

This is the part to obsess over. Every gift you send introduces a new person to Givin. Design the loop explicitly:

```
1. Ivan sends Jenny a free gift  →
2. Jenny gets a beautiful email, redeems in 1 click  →
3. Redeem page: "Gift someone back — your first one's free too"  →
4. Jenny signs up, sends to 2 people  → loop repeats
```

Levers that raise the viral coefficient (K): make redeeming delightful and instant, prompt the gift-back at the peak emotional moment, keep first-send free, and let every gift carry a tasteful "sent with Givin". If each new user invites just over one more who activates, you grow without paid marketing — which is the whole point of "lowest cost possible."

08 Data model (starting point)

```
users       id, linkedin_sub, name, email, photo, created_at
recipients  id, owner_id, name, email, headline, note, key_dates[]
gifts       id, slug, title, type, value, cost, tier(free|paid), active
orders      id, sender_id, recipient, gift_id, message, occasion,
            status(draft|scheduled|sent|redeemed), send_at, redeemed_at,
            tremendous_id, stripe_id, referred_from_order_id
credits     id, user_id, free_remaining, period, source(signup|sponsor)
user_roles  user_id, role(user|admin)  // separate table; never in users
```

Protect every table with Supabase Row-Level Security so a user can only read their own data. Keep `user_roles` in its own table (as your earlier admin build correctly did) to prevent privilege escalation.

09 Phased roadmap

PHASE 0 · MVP — weeks 1–4

Send one real free gift, end to end

LinkedIn login → pick from ~4 gifts → enter recipient email + note → Tremendous sends it → recipient redeems. Add a basic admin to manage the catalog. **Goal: prove the magic moment works.** Monthly cost: ~€0 + gift value.

PHASE 1 · THE LOOP — weeks 5–8

Make it spread

Gift-back prompt on the redeem page, first-send-free credits, "sent with Givin," saved recipients & key dates, scheduled delivery, paid gifts via Stripe. **Goal: K > 1 with a pilot group.**

PHASE 2 · EXTENSION — weeks 9–12

The thin Chrome launcher

"Send a gift" button on a profile, reads only the visible name/headline on click, opens Givin pre-filled. Ship to the Chrome Web Store. **Goal: cut send time to seconds where the moment happens.**

PHASE 3 · TEAMS — months 4–6

Where the money is

Calendar sync, team budgets & approvals, analytics, sponsored free gifts, Nordic provider for margin. **Goal: recurring B2B revenue.**

10 Security & privacy

- **Secrets in env vars only** — Stripe, Tremendous, Resend keys live in Supabase/Vercel secret stores, never in the repo or docs. (See the rotate-key alert above.)
- **Row-Level Security on every table**, server-side role checks, input validation (Zod). Your prior build's security model is right — keep it.
- **GDPR (you're in Sweden/EU):** minimize data, store recipient emails only with a lawful basis, give clear consent + deletion, and a plain privacy policy. Recipients must be able to opt out of future gifts.
- **Webhooks signed & verified** (Stripe, Tremendous) so nothing fakes a "paid" or "redeemed" event.

11 Legal & IP — straight talk

LinkedIn terms

Use LinkedIn only for login and as the place the user sees a moment. Don't scrape, bulk-read connections, automate actions, or message through it. The "companion launcher" rule in §03 keeps you compliant and keeps your users safe.

The patent

Be realistic: "send a scheduled gift card triggered by a social profile event" likely faces prior art, and EU software patents are narrow. A filing can have defensive and fundraising value, but **don't let it gate your build or your launch.** Your durable moat is brand, the viral loop, partner deals, and speed — not a patent. If you do file with PRV, file before any public disclosure.

GitHub vs. patent (your open question)

You chose "private repo now, decide later" — good. Keep the repo **private**. Public disclosure on GitHub before filing can destroy patent novelty in much of the world. Decide the patent question deliberately; until then, private is the safe default and costs you nothing.

Trademark & brand

"Givin" is a clean, ownable name. The original logo in the brand file deliberately **does not** borrow LinkedIn's "in" mark or colors — the earlier pink "Gift in" lockup would invite a trademark dispute and tie your brand to a platform you don't control. Register "Givin" as a word mark (PRV or EUIPO) and secure the domain + handles.

12 What it costs to run

Stage	Fixed monthly	Variable
MVP / pilot	~€0	Gift value per free send (~€2–4)
Early growth	~€25–50 (Supabase/Resend paid tiers)	Gift value + Stripe fees on paid gifts
Scaling	Grows with usage; still usage-based	Offset by sponsors + margin + plans

The only meaningful early cost is the free gifts themselves — and that's the budget you'd otherwise spend on ads, except here it buys both acquisition and delight. Cap it, and ideally sponsor it.

13 Repo structure for GitHub

```
givin/
├─ apps/
│  ├─ web/          React + Vite app (givin.app)
│  └─ extension/    Chrome MV3 launcher
├─ supabase/
│  ├─ migrations/   schema + RLS policies
│  └─ functions/    send-gift, redeem, webhooks
├─ packages/ui/    shared brand components
├─ .env.example     documented, NO real secrets
└─ README.md

# private repo · secrets via Supabase/Vercel env, not git
```

When your team is ready, I can scaffold this repo for you — the schema, the send/redeem Edge Functions, the React screens from the brand file, and the extension stub — so you can push it straight to a private GitHub repo and start filling in keys.

14 Decisions I need from you

1. **Rotate the Stripe key** (today).
2. **Free-gift funding:** self-fund the small reward, or pursue a sponsor (coffee/flower brand) from the start?
3. **First market:** launch global via Tremendous, or Nordic-first to court GoGift/Awardit early?
4. **Patent:** file with PRV before any public code, or deprioritize and stay fast + private?
5. **Should I scaffold the GitHub repo next** (web app + Supabase functions + extension stub), so your team has a running start?

Companion to this document: **"Givin Brand & UI"** — the original logo, the Chrome-extension dropdown, and the web-app screens, all in the brand above.